

# P1247R0

## Disabling static destructors: introducing `no_destroy` and `always_destroy` attributes

Published Proposal, 2018-10-08

**This version:**

<http://wg21.link/p1247r0>

**Authors:**

[Bruno Cardoso Lopes](#) (Apple)

[JF Bastien](#) (Apple)

[Erik Pilkington](#) (Apple)

**Audience:**

EWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Source:**

[github.com/bcardosolopes/cpp-papers/blob/master/p1247r0.bs](https://github.com/bcardosolopes/cpp-papers/blob/master/p1247r0.bs)

---

# Table of Contents

## 1 Introduction

## 2 Proposed Solution

## 3 Alternatives

3.1 `__cxa_atexit` override

3.2 use `std::quick_exit`

3.3 define a type yourself

3.4 Use a global reference

## 4 Future directions

### References

Informative References

## § 1. Introduction

The destruction of variables with static or thread local storage duration can introduce bugs and surprising behavior that is hard to anticipate, and isn't fully supported across all implementations of C++. Some of the issues include:

1. Teardown ordering: the order of destruction across translation units isn't always the same.
2. Multithreaded teardown: crashes when the main thread is exiting the process and calling destructors while a detached thread is still running and accessing the destructing variables.
3. Shared code that is compiled both as an application and as a library. When library mode is chosen, the same problems as above arise.
4. The operating system can reclaim many resources, particularly memory, faster than the user can anyways.
5. In embedded platforms, it's often the case that we know that static destructors are never called. It would be useful to avoid emitting them entirely to reduce the amount of generated code. Additionally, providing a way to annotate this property inline in the source code improves readability on these platforms.

6. `[basic.start.main]` currently says:

It is implementation-defined whether a program in a freestanding environment is required to define a `main` function. [ *Note*: In a freestanding environment, start-up and termination is implementation-defined; start-up contains the execution of constructors for objects of namespace scope with static storage duration; termination contains the execution of destructors for objects with static storage duration. — *end note* ]

This is an unfortunate language mode which we could eventually do away with if we required users of freestanding implementations to annotate this property into their source code.

Because of these issues, some implementations provide extensions for their users to disable static or thread local destructors. These extensions are broadly useful, but non-portable, and fragment the language to solve what should be a simple problem. Standardizing these extensions would benefit the entire C++ ecosystem.

## § 2. Proposed Solution

Add new attributes to enable/disable registration of exit-time destructors of static and thread storage duration variables.

The `no_destroy` attribute specifies that a variable with static or thread storage duration will not have its exit-time destructor run. It also implies that the destructor is not considered *potentially-invoked*, so it isn't odr-used, nor is it required to be accessible. For example:

```
struct widget
{
private:
    ~widget();
};

[[no_destroy]] widget w; // not an error!
```

Conversely, the `always_destroy` attribute specifies that a variable with static or thread storage duration should have its exit-time destructor run. This is the default behavior and is needed to allow users to opt-out of `no_destroy` when a compiler flag makes it the default (or when disabled at module level, see [\[p1245r0\]](#)).

These attributes currently have an [\[Implementation\]](#) in [\[Clang\]](#). This proposal is an effort to standardize existing practice.

More background and other possible solutions for this topic can be found in the clang mailing list thread [\[CFE2016\]](#) and [\[CFE2018\]](#). We considered other alternatives as part of the research for this paper. The [Alternatives](#) section provides insights why those solutions aren't sufficient.

## § 3. Alternatives

Here is a description of different solutions and why they're inadequate to solve this problem.

### § 3.1. `__cxa_atexit` override

Some projects currently override `__cxa_atexit` to avoid calling static destructors. This outright disables destructor calls for all static variables, not just the ones that the programmer has verified are safe. This is also non-portable, as `__cxa_atexit` is only present in the Itanium ABI.

### § 3.2. use `std::quick_exit`

This requires controlling all exit paths from a program, and like the [above](#) outright disables destructor calls for all static or thread variables, not ones that the programmer has verified are safe.

### § 3.3. define a type yourself

For instance, one could define `no_destroy` as a template:

```
template <class T> class no_destroy
{
    alignas(T) unsigned char data_[sizeof(T)];
public:
    template <class... Ts> no_destroy(Ts&&... ts)
    { new (data_) T(std::forward<Ts>(ts)...); }

    T &get() { return *reinterpret_cast<T *>(data_); }
};

no_destroy<widget> my_widget;
```

The class template `no_destroy` disables destruction by circumventing the type system, storing the value opaquely in a buffer. This does work, but technically has object lifetime issues. This also needlessly inhibits `constexpr` initialization, leading to poor performance. `no_destroy` can also disable the destructors of variables with automatic storage, which is an unavoidable misfeature of this design.

### § 3.4. Use a global reference

For instance:

```
widget&& w = *new widget();
```

Like the [above](#), this can work with variables with automatic storage, and disables `constexpr` initialization. This also adds needless pointer indirection.

## § 4. Future directions

In the future, we could move to make `[[no_destroy]]` the default for variables with static or thread storage. After these attributes have been standard for a long time, we could deprecate implicit `[[always_destroy]]` on static or thread variables with non-trivial destructors, then change the default.

## § References

### § Informative References

#### [CFE2016]

[Suppress C++ static destructor registration](http://lists.llvm.org/pipermail/cfe-dev/2016-July/050040.html). URL: <http://lists.llvm.org/pipermail/cfe-dev/2016-July/050040.html>

#### [CFE2018]

[Suppress C++ static destructor registration, take 2](http://lists.llvm.org/pipermail/cfe-dev/2018-July/058494.html). URL: <http://lists.llvm.org/pipermail/cfe-dev/2018-July/058494.html>

#### [Clang]

[Attributes in Clang](http://clang.llvm.org/docs/AttributeReference.html#no-destroy). URL: <http://clang.llvm.org/docs/AttributeReference.html#no-destroy>

#### [Implementation]

[Clang implementation](https://gcc.godbolt.org/z/xCh1jW). URL: <https://gcc.godbolt.org/z/xCh1jW>

**[P1245R0]**

Bruno Cardoso Lopes; JF Bastien. [export module containing \[\[attribute\]\]](#); URL:  
<http://wg21.link/p1245r0>